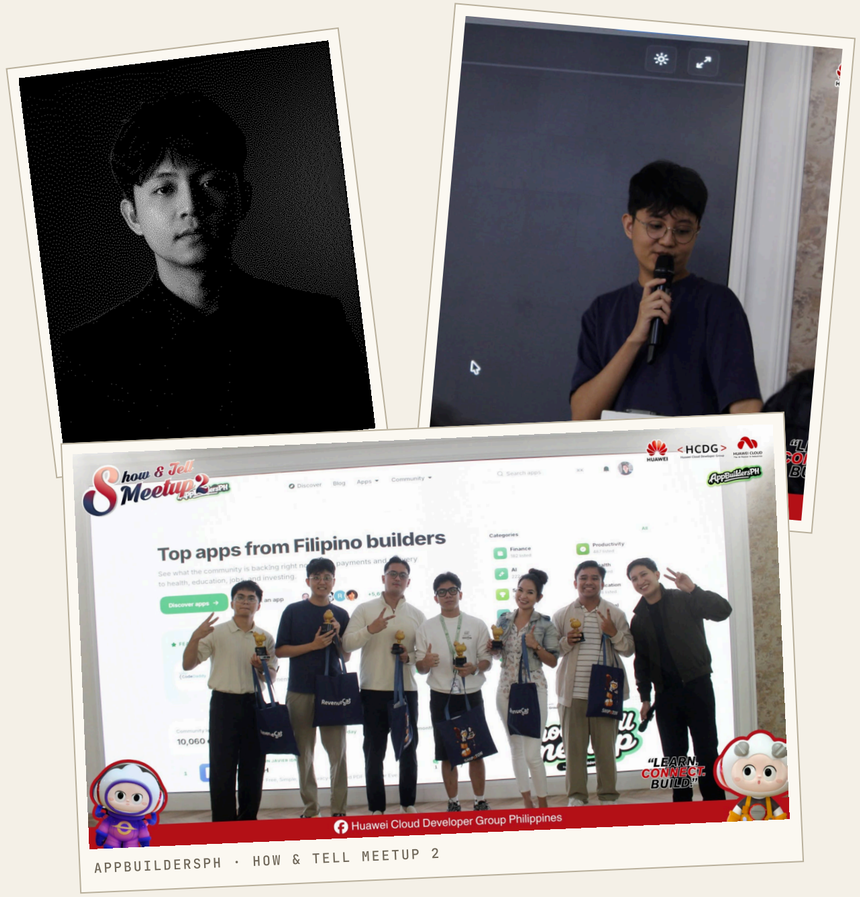
INNOVATEHACK+ 2.0 · GDG ON CAMPUS XAVIER ATENEO

From Idea to MVP

Scoping, Stack Selection, and Building with Google AI Tools



APPBUILDERSPH · HOW & TELL MEETUP 2

SPEAKER

Arron Kian Parejas

AI Engineer

DATE

November 16, 2026

Google Meet

FORMAT

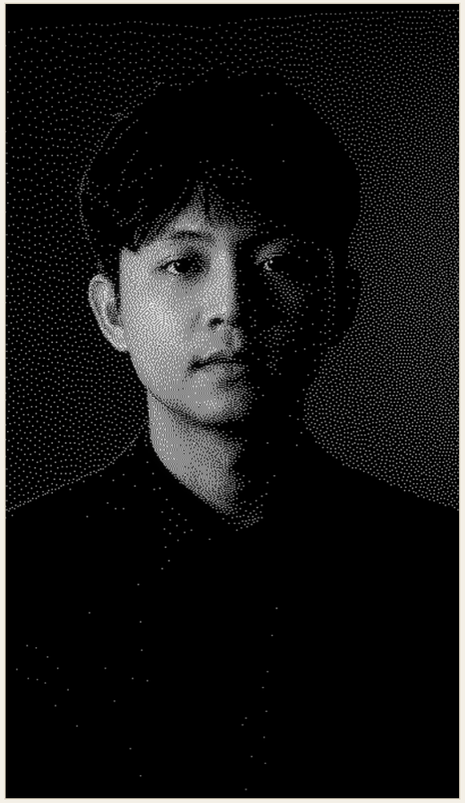
45 minutes

15 slides · bring your own idea

FROM IDEA TO WORKING PRODUCT

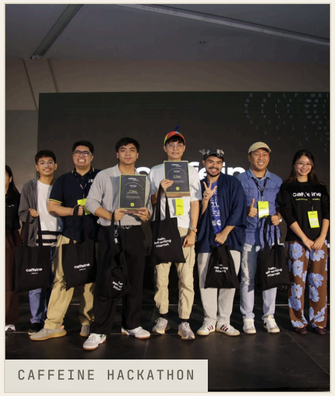
MEET YOUR SPEAKER

A builder. An engineer. An open-source contributor.



A practical method for turning an idea into working software.

01	AI Engineer
02	ML Engineering Intern @ FlyRank.ai
03	Founder of DomoDomo
04	Builder of Helios, an open-source language model
05	Former GDG on Campus HAU Chapter Lead
06	Open-source contributor
07	Ship-A-Little winner, Shipaton 2026



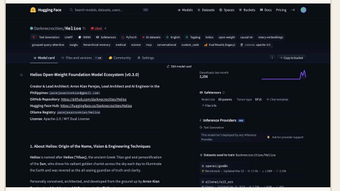
CAFFEINE HACKATHON



APPCON 2024



SKYDEV



HELIOS ON HUGGING FACE

Top 1
APPBUILDERSPH

Top 7
GITHUB · PHILIPPINES

7
HACKATHON WINS

182+
PROJECTS

THE HACKATHON MINDSET

Great ideas don't automatically become great products.

01 IDEA → 02 EXECUTION → 03 VALIDATION → 04 PRODUCT

The idea is the easy part. What separates a demo from a product is the discipline between them.

01 **Overengineering**
A perfect system nobody can demo.

02 **Technology before requirements**
Picking the stack before knowing the problem.

03 **Shipping untested code**
Features that fail live on demo day.

START WITH THE PROBLEM

Understand the problem before choosing the technology.

WHO

Who are we helping?

A specific person, not "everyone".

WHAT

What problem are we solving?

One pain point you can observe.

WHY

Why does it matter?

The cost of leaving it unsolved.

VAGUE IDEA

"Let's build an app for disasters."

- Who? Everyone.
- Problem? "Disasters".
- Why? Because it's a good idea.

→

CLEAR PROBLEM STATEMENT

Barangay residents in flood-prone areas can't report emergencies fast enough, so responders lack accurate locations. We help them send a located report in under 30 seconds.

WHO	Residents of flood-prone barangays
WHAT	A located report in under 30 seconds
WHY	Responders lose time and act on the wrong location

Illustrative example for this talk.

DEFINE YOUR MVP

Build less. Deliver more.

EMERGENCY REPORTING PLATFORM

Suggested split

Clear board

ALL FEATURES SORTED — CLEAR THE BOARD TO START AGAIN

Must Have 3

One-tap emergency report	Must
Automatic GPS location	Must
Responder dashboard	Must

Should Have 3

Photo attachment	Should
Live report status	Should
Push notifications	Should

Could Have 2

Voice reporting	Could
Multi-language support	Could

Won't Have · Deferred 2

Analytics dashboard	Won't
Social media sharing	Won't

DESIGN BEFORE YOU CODE

Prototype the experience first.

01
PROMPT
Describe the screen

→ 02
DESIGN
Generate UI options

→ 03
PROTOTYPE
Click through the flow

→ 04
IMPLEMENT
Build in code

GOOGLE STITCH

From prompt to UI

Turn a text description into screen designs you can iterate on.

stitch.withgoogle.com ↗

MATERIAL DESIGN

Components and tokens

Proven patterns for color, shape, motion, and accessibility.

m3.material.io ↗

GOOGLE FONTS

Type that ships

Free, fast, open-source fonts for web and mobile apps.

fonts.google.com ↗

TYPE SPECIMEN — INTER

Aa

Report an emergency



16/24 · 4.5:1 CONTRAST

Send report

CHOOSE YOUR TECH STACK

The right tool depends on the problem.

Try the stack selector (S)

What does your problem need?

Mobile	→	**Flutter** — One codebase for Android and iOS, with fast UI iteration.
Backend	→	**Firebase** — Managed Auth, Firestore, and Hosting without running servers.
Generative AI	→	**Gemini** — Text, image, and structured output via the API or Firebase AI Logic.
Computer Vision	→	**ML Kit** — Ready-made on-device APIs for text, barcode, and face detection.
Offline AI	→	**Google AI Edge** — Run models on the device with LiteRT, no connection needed.
Custom Infrastructure	→	**Google Cloud** — Cloud Run, Vertex AI, and BigQuery for custom services at scale.

THE GOOGLE DEVELOPER TOOLKIT

A toolkit for modern builders.

DESIGN

Design

Describe a screen and get editable designs, then build on proven components.

Stitch | Material Design | Google Fonts

DEVELOPMENT

Development

One codebase for Android, iOS, and web, with a fast UI loop.

Flutter

BACKEND

Backend

Auth, database, storage, and functions without running servers.

Firebase

GENERATIVE AI

Generative AI

Prompt, test, and ship generative models from a client or a backend.

Gemini | Google AI Studio | Firebase AI Logic

MOBILE ML

Mobile ML

Ready-made on-device APIs, or run your own model with LiteRT.

ML Kit | Google AI Edge | LiteRT

CLOUD

Cloud

Custom services, model training, and analytics at scale.

Google Cloud | Cloud Run | Vertex AI | BigQuery

OFFICIAL DOCS

Gemini API ↗ | AI Studio ↗ | Firebase AI Logic ↗ | ML Kit ↗ | Google AI Edge ↗ | Flutter ↗ | Firebase ↗ | Cloud Run ↗ | Stitch ↗ | Free Stack Handbook ↗

BUILD WITH GOOGLE AI

Give your application intelligence.

01 APPLICATION → 02 AI SERVICE → 03 VALIDATION → 04 RESPONSE

01
Gemini API
Call Gemini models from your backend.
Docs ↗

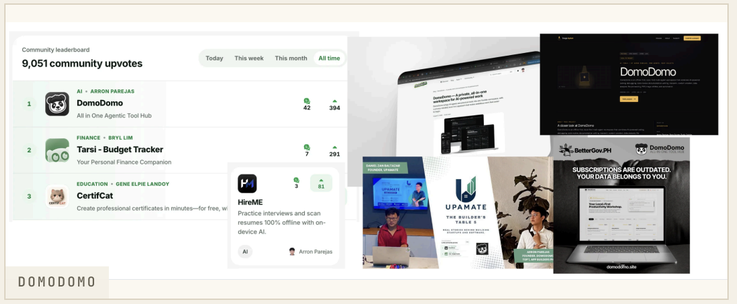
02
Firebase AI Logic
Call Gemini from mobile and web apps.
Docs ↗

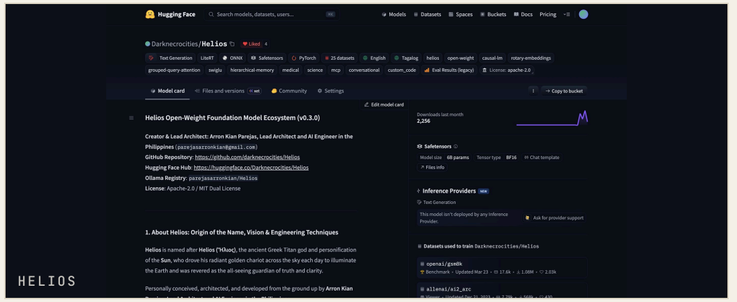
03
Function calling
The model requests; your code acts.
Docs ↗

04
Structured outputs
JSON that matches your schema.
Docs ↗

05
Multimodal AI
Text, images, audio, and video in one request. Always validate what comes back.
Docs ↗

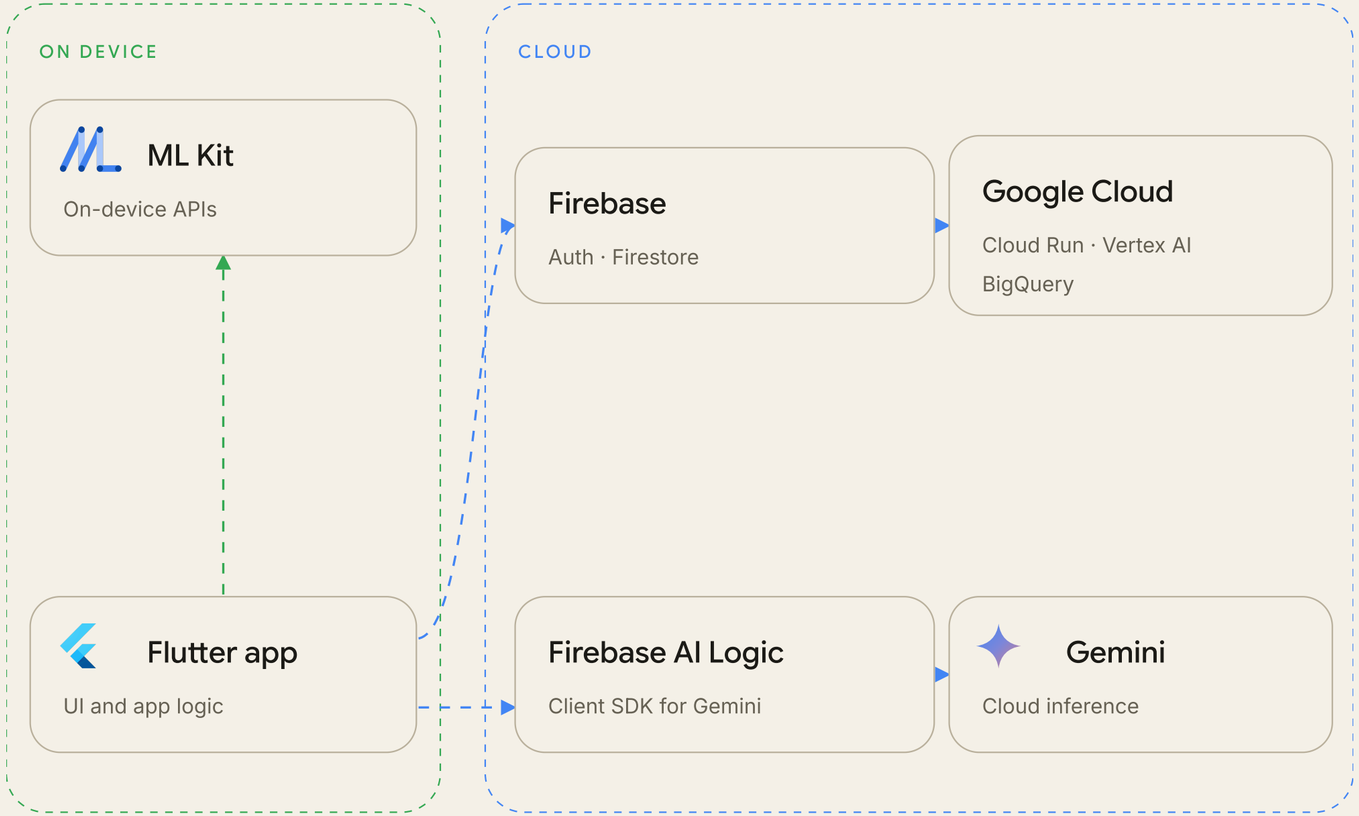






CLOUD AI VS ON-DEVICE AI

Where should your AI run?



	CLOUD	ON-DEVICE
Latency	Network round trip	Immediate, local
Connectivity	Internet required	Works offline
Privacy	Data leaves the device	Data stays local
Constraints	Usage costs and quotas	Memory, battery, model size

CLOUD · GEMINI API
DEVICE · ML KIT / AI EDGE / LITERT

Helios ships LiteRT and ONNX tags on Hugging Face — on-device formats matter for real projects.

PROMPTING IS ENGINEERING

Better instructions. Better engineering.

WEAK PROMPT

"Build me an AI-powered application."

- × No context, so you get generic code
- × No constraints, so it overengineers
- × No definition of done, so you cannot test it

STRUCTURED PROMPT

```
Context: Flutter app for barangay flood reports.
Constraints: Firebase backend, 3-day sprint,
  works on low-end Android.
Expected outcome: User submits a located
  report in under 30 seconds.
Architecture: Flutter UI, Firestore,
  Gemini via Firebase AI Logic for triage.
Testing: Unit tests for validation,
  one end-to-end submit flow.
```

Illustrative prompt for this talk.

THE CLEAR FRAMEWORK

Structure your prompts.

Reveal next (→ / R) | Show all | Reset

C

Context

Who is it for and what exists already?

Without context, the model guesses at your product.

Flutter app for barangay flood reports.

L

Limitations

Time, budget, stack, devices.

Limits are what stop the output from overengineering.

3-day sprint. Firebase. Low-end Android.

E

Expected outcome

A result you can check.

A checkable outcome is what you can write a test against.

Located report sent in under 30 seconds.

A

Architecture

How the pieces connect.

A diagram beats a paragraph when you mean 'these two'.

Flutter UI → Firestore → Gemini triage.

R

Requirements

Tests and quality bars.

Requirements survive the next person who touches the code.

Validation unit tests + one end-to-end flow.

PROMPT SO FAR

```
Context: Flutter app for barangay flood reports.
Limitations: 3-day sprint. Firebase. Low-end Android.
Expected outcome: Located report sent in under 30 seconds.
Architecture: Flutter UI → Firestore → Gemini triage.
Requirements: Validation unit tests + one end-to-end flow.
```

SET UP YOUR TEAM

Good engineering starts with good foundations.

01 **GitHub repository**
One shared source of truth.

02 **Feature branches**
One branch per task.

03 **Pull requests**
Review before merging.

04 **Environment variables**
Keys stay out of Git.

05 **Shared documentation**
README, setup, decisions.

06 **Task ownership**
Every task has one owner.

```
my-mvp/
├── lib/            # app code
├── test/           # unit tests
├── docs/           # decisions, setup
├── .env.example    # names only, no secrets
├── .gitignore      # ignores .env
└── README.md

main        stable, always demo-ready
└ feat/report-form   → PR → review → merge
└ feat/triage-ai     → PR → review → merge

PR checklist
  [ ] branch named feat/…
  [ ] tests pass locally
  [ ] README updated
  [ ] no secrets committed
```

FROM SPRINT TO SHIP

Every sprint needs a deliverable.

01 IDEATION → 02 SETUP → 03 BUILD → 04 TEST → 05 DEPLOY

DAYS 1–2

Feature development

Build the one core flow end to end.

DELIVERABLE — ONE WORKING FLOW

DAY 2–3

Integration testing

Connect UI, backend, and AI.

DELIVERABLE — END-TO-END RUN

DAY 3

Bug fixing

Fix what breaks the demo first.

DELIVERABLE — A GREEN DEMO

DAY 3–4

Demo preparation

Rehearse, record a backup, deploy.

DELIVERABLE — DEPLOYED AND RECORDED

DEFINITION OF DONE

- One flow you can demo in 60 seconds
- No secrets in the repository
- A recorded backup video
- A README that says how to run it

THANK YOU · Q&A

Build something that matters.

- Understand the problem.
- Choose the right tools.
- Engineer the solution.
- Validate everything.
- Ship something that works.



SCAN TO OPEN

FREE DEVELOPER STACK HANDBOOK

200+ free & open-source tools

A link-rich directory of Google technologies, APIs, AI, UI/UX, and full-stack development.

Open the handbook ↗



SOCIALS

Arron Kian Parejas

AI Engineer

LinkedIn ↗

Helios on GitHub ↗

Helios on Hugging Face ↗

DomoDomo ↗